



Laboratory Activity No. 2

From Syntax to Semantics: Building a Grammar Checker and Evaluator

Name(s):	Section:
Agarin, Justine Paul E.	BSCS 4-3
Gabuyo, Kevin E.	
Marjalino, Crislance James F. Marjalino	
Sanone, Lei Marvin Sanone	
Group No. / Team:	Date:
	09-16-2026

Every valid program answers two questions at once: is it built correctly (syntax), and what does it actually do (semantics)? In this activity, you will write the two pieces of software that answer those questions for a small language of arithmetic expressions — one that checks the rules, and one that carries out the meaning.

I. Learning Objectives

By the end of this activity, students should be able to:

- Translate a formal grammar (BNF and EBNF) into an algorithm that recognizes valid sentences of a language.
- Distinguish, in a working program, between checking that an input is syntactically valid (syntax) and computing what it means (semantics).
- Represent a recursive-descent parsing algorithm as a flowchart.
- Demonstrate, with a concrete example, how an ambiguous grammar produces inconsistent meaning, and how precedence and associativity rules resolve it.

II. Background: The Grammar You Will Implement

You will build a recognizer and evaluator for simple arithmetic expressions over single-digit integers, using + , - , * , / and parentheses. The grammar is given below in both BNF and EBNF so you can see they describe exactly the same language (the EBNF shorthand discussed in lecture).

Quick reminder before you read the grammar: anything written in angle brackets, like <expr>, is a nonterminal — a placeholder category that still needs to be expanded, and never appears in the final input string itself. Everything else (digits, + , - , * , / , parentheses) is a terminal — an actual character your program will read. Your recognizer's job is essentially to keep expanding nonterminals in its head until it either matches your input exactly (accept) or hits a contradiction (reject).

BNF:

```
<expr>  -> <expr> + <term> | <expr> - <term> | <term>
<term>  -> <term> * <factor> | <term> / <factor> | <factor>
<factor> -> ( <expr> ) | <digit>
<digit> -> 0|1|2|3|4|5|6|7|8|9
```

EBNF (equivalent):

```
<expr>  -> <term> { ( + | - ) <term> }
<term>  -> <factor> { ( * | / ) <factor> }
<factor> -> ( <expr> ) | <digit>
<digit> -> 0|1|2|3|4|5|6|7|8|9
```



Notice this grammar already has precedence and associativity built in — $*$ and $/$ bind tighter than $+$ and $-$ because $\langle \text{term} \rangle$ sits below $\langle \text{expr} \rangle$ in the hierarchy, and all four operators are left-associative because of how the repetition unfolds left to right. This is the corrected, unambiguous version of the grammar, with precedence and associativity already fixed.

III. Part 1 — Flowchart the Recognizer

Before writing any code, design a flowchart that traces how a recursive-descent recognizer decides whether an input string belongs to the language defined above. Your flowchart must show, at minimum:

- The three levels of the grammar (expr , term , factor) as distinct stages or subprocesses.
- The decision points where the next token determines which rule applies (for example: is the next character a digit, an operator, or a parenthesis?).
- The path that leads to ACCEPT (input fully consumed, no rule violated) and the path(s) that lead to REJECT (unexpected token, or leftover input after the expression ends).
- Where recursion occurs (e.g., a factor calling back into expr when it encounters an opening parenthesis).

You may use any flowcharting tool (Draw.io, Lucidchart, Canva, or hand-drawn and scanned), but it must be legible and use standard flowchart symbols (terminal, process, decision, connector).

IV. Part 2 — Implement the Checker and Evaluator

Using any programming language you are comfortable with (Python is recommended), implement a program with two required behaviors for every input string:

Task A — Syntax Check (the Recognizer):

- Read the input string and determine whether it is a valid sentence of the grammar in Section II.
- If it is NOT valid, print: Invalid syntax, along with the position or token where the check failed.

Task B — Semantic Evaluation (the Meaning):

- If — and only if — the input passes Task A, compute and print the numeric value the expression evaluates to, following normal precedence and left-associativity.
- This is operational semantics in miniature: the “meaning” of the expression is defined by carrying out the computation, the same way operational semantics defines meaning as “what changes in the machine’s state as each step executes.”

Stretch Requirement — Show the Villain, Then Fix It:

Before using the corrected grammar above, first implement a naive evaluator against this ambiguous, unparenthesized rule: $\langle \text{expr} \rangle \rightarrow \langle \text{expr} \rangle + \langle \text{expr} \rangle \mid \langle \text{expr} \rangle * \langle \text{expr} \rangle \mid \langle \text{digit} \rangle$. Using the input $2+3*4$, show that a left-to-right-only evaluation gives 20 while the mathematically intended precedence gives 14. Include both outputs as a comment or printed comparison in your submission, then explain in one paragraph how the grammar in Section II resolves this ambiguity.

V. Required Test Cases

Run your program on each input below and record the output in your submission.

Input String	Expected Syntax Result	Expected Value (if valid)
3+4*2	Valid	11
(3+4)*2	Valid	14
8/2-1	Valid	3
3++4	Invalid	—
(3+4	Invalid (unbalanced parenthesis)	—
2+3*4	Valid	14 (must NOT be 20)

VI. Grading Rubric

Criterion	Description	Points
Flowchart	Accurately represents the recursive-descent logic ($\text{expr}/\text{term}/\text{factor}$ levels, decision points, accept/reject paths)	20

Syntax Checker (Task A)	Correctly accepts valid and rejects invalid strings, including the required test cases	25
Semantic Evaluator (Task B)	Correctly computes values with proper precedence and associativity	25
Ambiguity Demonstration	Clearly shows the $2+3*4$ discrepancy and explains the fix	15
Code Quality & Comments	Readable, organized, and commented to show the connection to grammar rules	10
Reflection Questions	Thoughtful, accurate answers connecting code to lecture concepts	5
TOTAL		100

VII. Reflection Questions

1. In your own words, what is the difference between what your Task A code checks and what your Task B code checks?
- The difference between the Task A is it checks the syntax of the code. It checks if the input format follows the rule of the BNF. The Task B code checks the semantics or rather the actual part where the arithmetic happens
2. Point to the exact place in your code where recursion happens, and explain which grammar rule it corresponds to.
- if not chars:

raise ValueError(f"unexpected end of input (position {len(original)})")

```
c = chars[0]
if c == "(":
    chars.pop(0)
    value = parse_expr(chars, original)
    if not chars or chars[0] != ")":
        pos = len(original) - len(chars)
        raise ValueError(f"expected ')' (position {pos})")
    chars.pop(0)
    return value
elif c in "0123456789":
    chars.pop(0)
    return int(c)
else:
    pos = len(original) - len(chars)
    raise ValueError(f"unexpected token '{c}' (position {pos})")
```

This code in particular is for the grammar rule: `<factor> -> ( <expr> ) | <digit>`

2. Explain, using your $2+3*4$ example, why an ambiguous grammar is a real problem and not just a theoretical one.
- Because $2+3*4$ presents 2 branches, one for $(2+3)*4$ and $2+(3*4)$. Depending on the implementation, it can result in 2 different answers, hence the ambiguous problem rather than a theory that isn't replicable.
4. If you were to add a new operator (for example, exponentiation $^$) to this grammar, where in the expr/term/factor hierarchy would it need to go, and why?
- It would be on the line after the operation/ function for multiplication and division. It takes precedence over them based on the PEMDAS rule, thus it would need to call them first to check before proceeding.

VIII. Submission Requirements

- Flowchart (PDF or PNG export).
- Source code file(s), with comments.
- A short output log or screenshots showing all required test cases running.
- Answers to the reflection questions (this document or a separate page).
- Submit all files as a single compressed folder named LastName_Lab_SyntaxSemantics.zip.
- Create a GitHub repository for your submission and push all required files to the repository.
- Name your repository: LastName_Lab_SyntaxSemantics

